

Жизненный цикл продукта

Комплект документации по проекту игры PORO

Документ подготовлен для задания на тему жизненного цикла продукта и включает концепт-документ, техническое задание и диаграмму классов.

Название проекта	PORO
Тип продукта	Одиночная консольная RPG с элементами roguelike
Платформа	ПК, запуск в терминале Windows/Linux/macOS
Цель проекта	Создать увлекательную пошаговую RPG с системой прокачки, боями с боссами, сундуками-головоломками и магазином улучшений
Итоговый артефакт	Рабочий прототип игры и комплект проектной документации

1. Концепт-документ проекта

- PORO — пошаговая консольная игра в жанре RPG, в которой игрок исследует подземелья, сражается с боссами, открывает сундуки с загадками, прокачивает персонажа, покупает улучшения в магазине и проходит 15 уровней сложности.
- Целевая аудитория: школьники, студенты и начинающие игроки, которым интересны RPG, логические задачи, прокачка персонажа и пошаговые стратегии.
- Проблема, которую решает продукт: большинство учебных RPG-проектов ограничиваются базовым перемещением и боем. Здесь предлагается более глубокая структура: система уровней, уникальные боссы, загадки, магазин, музыкальное сопровождение и сохранение прогресса.
- Игровая цель пользователя: пройти 15 уровней, победить всех боссов, собрать максимальное количество монет, улучшить персонажа и завершить игру.
- Ключевая ценность продукта: сочетание RPG-механик, головоломок, стратегии и прокачки в простом консольном интерфейсе без сложной графики.

Основные механики

- Исследование карты размером 32x16 клеток и переход между уровнями.
- Пошаговая боевая система с боссами (15 уникальных противников).
- Система критических ударов и уворота с расчётом вероятности.
- Сундуки с загадками (30 уникальных головоломок).
- Магазин с 6 видами улучшений (HP, урон, зелья, крит, броня, уворот).
- Система уровней и опыта с 5 вариантами прокачки.
- Музыкальное сопровождение (фоновое воспроизведение MP3).
- Инвентарь с зельями лечения.
- Ведение лога ключевых игровых событий.

Этапы жизненного цикла продукта

- Инициация: формулировка идеи, определение жанра, сбор требований и анализ похожих RPG-игр.
- Планирование: описание функциональных и нефункциональных требований, проектирование сущностей и игровых сценариев.
- Проектирование: подготовка концепта, технического задания и диаграммы классов.
- Разработка: реализация игровых сущностей, логики перемещения, боевой системы, головоломок, магазина, системы прокачки, музыки и сохранений.
- Тестирование: проверка всех сценариев прохождения, балансировки боссов, корректности работы загадок и сохранений.
- Сопровождение: исправление ошибок, балансировка сложности, добавление новых уровней и механик.

2. Техническое задание на разработку

2.1 Назначение системы

Разрабатываемый программный продукт представляет собой консольную игру в жанре пошаговой RPG. Система должна позволять игроку перемещаться по карте, сражаться с боссами, решать головоломки, прокачивать персонажа и проходить игру до финального 15-го уровня.

2.2 Функциональные требования

- Система должна содержать 15 уровней, каждый со своей уникальной картой и боссом.
- Первый уровень является стартовым и содержит базового босса.
- Последний (15-й) уровень является финальным с самым сильным боссом.
- На каждом уровне должен быть как минимум один сундук с загадкой.
- Должно быть не менее 30 уникальных загадок с ответами.
- Должны использоваться как минимум 2 типа улучшений персонажа (НР и урон).
- В игре должен присутствовать босс на каждом уровне (15 уникальных боссов).
- Каждая карта должна отображаться в терминале с цветовой кодировкой.
- Каждый сундук открывается только после решения загадки.
- При неправильном решении загадки сундук остаётся закрытым (без штрафа).
- Игрок должен иметь инвентарь с зельями и возможность их использовать.
- Система должна обеспечивать сохранение прогресса между уровнями.
- Все ключевые действия пользователя должны выводиться на экран.
- Система должна поддерживать фоновое воспроизведение музыки.
- Должен быть реализован магазин с 6 видами улучшений за монеты.

2.3 Нефункциональные требования

- Интерфейс должен быть понятным и полностью текстовым, с цветным выделением элементов.
- Код программы должен быть модульным и поддерживать расширение количества уровней, боссов и загадок.
- Частота обновления экрана должна составлять 30 FPS.
- Приложение должно корректно работать на Windows/Linux/macOS с компилятором C++11 и выше.
- Ошибки ввода пользователя не должны приводить к аварийному завершению игры.
- Музыка должна корректно останавливаться при выходе из игры.

2.4 Основные сущности системы

- Player: хранит текущее здоровье, максимальное здоровье, урон, уровень, опыт, количество зелий, монеты, шанс крита, броню, шанс уворота и текущую позицию на карте.

- Boss: хранит имя, здоровье, максимальное здоровье, урон, броню, шанс крита, состояние жизни, координаты и параметры AI (шанс атаки, кулдаун).
- Chest: хранит координаты, состояние открытости, загадку и ответ на неё.
- Game: управляет всем игровым процессом: генерацией карт, инициализацией боссов, музыкой, рендерингом, обработкой ввода и состоянием игры.
- Puzzle: проверяет ответ игрока на загадку.
- Shop: предоставляет 6 видов улучшений за монеты с разной стоимостью.

2.5 Пользовательские сценарии

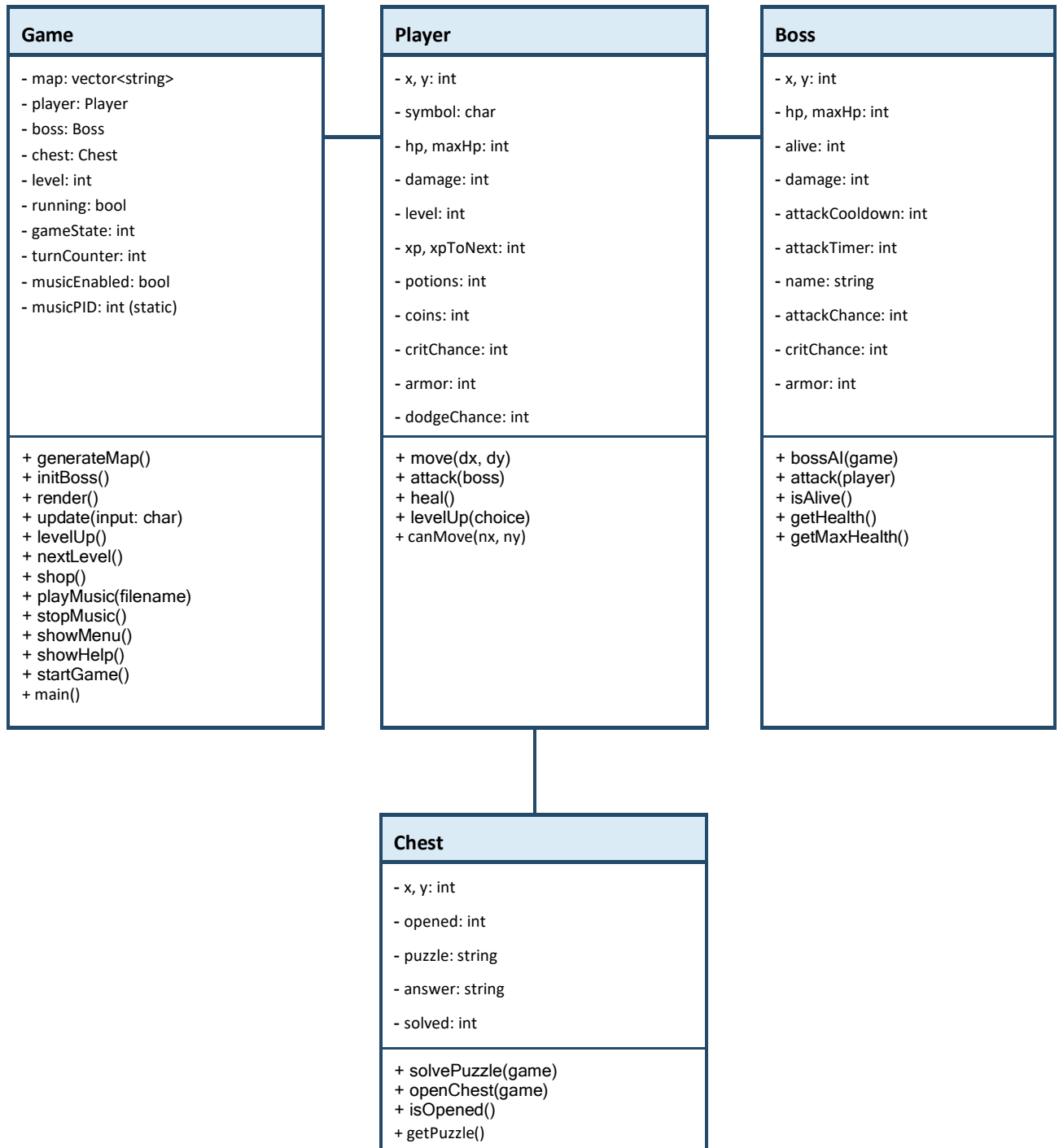
- Игрок запускает игру, видит главное меню с музыкой и выбирает "Начать игру".
- Игрок перемещается по карте (WASD), находит босса и атакует его (пробел).
- Игрок находит сундук (E), решает загадку и получает награду.
- При победе над боссом игрок получает опыт и монеты.
- Игрок заходит в магазин и покупает улучшения за монеты.
- Игрок повышает уровень и выбирает одно из 5 улучшений.
- Игрок переходит на следующий уровень с новым боссом.
- Игрок может включить/выключить музыку (M).
- Игрок сохраняет прогресс между уровнями и продолжает игру позже.

2.6 Критерии приёмки

- Все обязательные сущности созданы и взаимодействуют без критических ошибок.
- Игру можно пройти от 1-го до 15-го уровня минимум по одному сценарию.
- При правильном ответе на загадку сундук открывается и выдаёт награду.
- Сундуки выдают предметы (HP, зелья, монеты, урон, уворот).
- Боссы атакуют игрока с заданной вероятностью.
- Магазин корректно работает и улучшает характеристики.
- Система прокачки работает при наборе опыта.
- Музыка воспроизводится и корректно выключается.

3. Диаграмма классов

Ниже представлена логическая UML-структура основных классов игры. Диаграмма показывает центральный управляющий класс Game, игровые сущности Player и Boss, а также связанные объекты: карту, сундуки, магазин и систему музыки.



Краткая интерпретация диаграммы

- Класс Game управляет всем жизненным циклом игровой сессии: генерацией карт, инициализацией боссов, рендерингом, обработкой ввода, музыкой и переходом между уровнями.
- Player связан с картой через координаты и использует инвентарь (зелья) для лечения, а также имеет систему прокачки.
- Boss содержит AI-логику, атакует игрока с заданной вероятностью и имеет свои характеристики (здоровье, урон, броня, крит).
- Chest содержит загадку и проверяет ответ игрока, выдавая награду при правильном решении.
- Shop реализован как функция внутри Game и предоставляет 6 улучшений.
- Music реализован через системные вызовы ОС для фонового воспроизведения MP3.

Вывод

Подготовленный комплект документации покрывает обязательные требования и содержит: техническое задание, концепт-документ и диаграмму классов проекта. Документ можно использовать как основу для защиты задания или как базу для последующей реализации игры.